

Projekt II - Przetwarzanie równoległe na kartach graficznych (CUDA)

Emilia Bonk 160108
Szymon Urban 160271

Wstęp.

- **Nazwa przedmiotu:** Laboratorium z przetwarzania równoległego
- **Autorzy:** Emilia Bonk (160108), Szymon Urban (160271)
- **Grupa dziekańska:** L09
- **Termin zajęć laboratoryjnych:** Wtorek 9:45-11:15
- **Opis zadania:** Celem ćwiczenia była analiza optymalizacji kafelkowego algorytmu mnożenia macierzy w środowisku CUDA z wykorzystaniem pamięci współdzielonej bloku wątków. Zadanie polegało na zbadaniu wpływu zwiększenia intensywności obliczeniowej (obliczanie 1, 2 oraz 4 wyników przez pojedynczy wątek) na efektywność przetwarzania.
- **Adres e-mail:** emilia.bonk@student.put.poznan.pl, szymon.urban@student.put.poznan.pl

1 Informacje techniczne.

Parametr systemu GPU	Wartość
Model karty graficznej	NVIDIA GeForce GTX 1080 (Architektura Pascal)
Liczba multiprocesorów (SM)	20 multiprocesorów
Liczba rdzeni CUDA ogółem	2560 rdzeni CUDA
Maksymalny zegar GPU	1734 MHz (1.73 GHz)
Pojemność pamięci globalnej VRAM	8112 MBytes (8 GB)
Maksymalna pamięć współdzielona na blok	49152 bytes (48 KB)
Wielkość pamięci współdzielonej multiprocesora	98304 bytes (96 KB)
Dostępna liczba rejestrów na blok	65536
Rozmiar warpu (wiązki)	32 wątki
Wersja sterownika / runtime CUDA	12.2 / 12.5

Tablica 1: Informacje techniczne dotyczące środowiska testowego GPU

2 Prezentacja i analiza kodów.

Uzasadnienie rozmiaru instancji problemu

Karta NVIDIA GeForce GTX 1080 posiada 20 multiprocesorów (SM). Każdy SM może przetwarzać maksymalnie 2 bloki po 1024 wątki, co daje 40 bloków wykonywanych w jednym momencie (tzw. pełna fala). Dla macierzy 3200×3200 i bloku 32×32 , siatka (Grid) składa się z $100 \times 100 = 10000$ bloków. Liczba ta jest 250 razy większa niż pojemność jednej fali (40 bloków), co gwarantuje ciągle nasycenie pracą wszystkich multiprocesorów przez cały czas trwania obliczeń i minimalizuje błędy pomiarowe na styku rozpoczęcia i zakończenia działania kernela.

Zgodnie z wymaganiami projektowymi przygotowano trzy warianty jądra (kernela) realizującego kafelkowe mnożenie macierzy z wykorzystaniem pamięci współdzielonej. Zbadano bloki wątków o rozmiarach 16×16 oraz 32×32 . W każdym przypadku wielkość przetwarzanej instancji problemu (macierze 3200×3200) dobrano tak, aby w pełni nasycić 20 multiprocesorów karty NVIDIA GeForce GTX 1080.

Teoretyczna analiza limitów przydziału pracy (Siatka wyników)

Zgodnie ze specyfikacją sprzętową karty NVIDIA GeForce GTX 1080, limit pamięci współdzielonej dostępnej dla pojedynczego bloku wątków wynosi dokładnie 49152 bajty (48 KB). Przy założeniu, że słowo danych typu `float` zajmuje 4 bajty, możemy teoretycznie wyznaczyć maksymalną liczbę wyników, jaką może obliczyć pojedynczy wątek w konfiguracjach bloków 32×32 oraz 16×16 :

- **Dla bloku 32×32 (1024 wątki):** Pojedynczy kafel danych z jednej macierzy zajmuje $32 \times 32 \times 4 \text{ B} = 4096$ bajtów. Dostępna pamięć współdzielona pozwala na jednoczesne załadowanie maksymalnie $49152/4096 = 12$ bloków danych. Przekłada się to na po 6 bloków z macierzy A i macierzy B. Teoretycznie pozwala to na obliczenie siatki $6 \times 6 = 36$ wyników częściowych przez pojedynczy wątek.
- **Dla bloku 16×16 (256 wątków):** Pojedynczy kafel danych zajmuje $16 \times 16 \times 4 \text{ B} = 1024$ bajty. Pamięć współdzielona bloku jest w stanie pomieścić aż $49152/1024 = 48$ bloków danych (po 24 bloki z macierzy A i B). Teoretycznie pozwala to jednemu wątkowi na wyznaczenie siatki aż $24 \times 24 = 576$ wyników częściowych.

W warunkach rzeczywistych te ekstremalne granice teoretyczne są niemożliwe do osiągnięcia z powodu ograniczeń fizycznego rozmiaru pliku rejestrów procesorów strumieniowych (SM), które uniemożliwiają przechowywanie tak ogromnej liczby zmiennych akumulacyjnych bez zjawiska *register spilling* do wolnej pamięci lokalnej.

Wariant 1: Oryginalny algorytm (1 wynik obliczany przez wątek)

Oznaczenie skrótowe: 1-Res

Wersja bazowa zaczerpnięta z przykładów NVIDIA. Każdy wątek ładuje do pamięci współdzielonej po jednym słowie z macierzy A i B, a następnie oblicza dokładnie jeden wynik cząstkowy tablicy C. Wątek przechowuje wynik tymczasowy w jednej zmiennej rejestrowej (`Csub`). Jak widać na Listingu 1 oraz w kodzie źródłowym `matrixMul.cu`, dostęp do pamięci globalnej w celu załadowania kafelków odbywa się w sposób koalescyjny.

```
1 // matrixMul.cu
2 As[ty][tx] = A[a + wA * ty + tx];
3 Bs[ty][tx] = B[b + wB * ty + tx];
```

Listing 1: Fragment jądra obliczającego 1 wynik na wątek.

Wątki w wiązce (tx = 0..31)

 ... tx=0..31 (1 wiązka)



Każda instrukcja: $32 \text{ wątki} \times 4 \text{ B} = 128 \text{ B} = 1 \text{ transakcja L2/DRAM} \checkmark$

Rysunek 1: Koalescencyjne pobieranie danych do pamięci współdzielonej w wariantcie 1-Res.

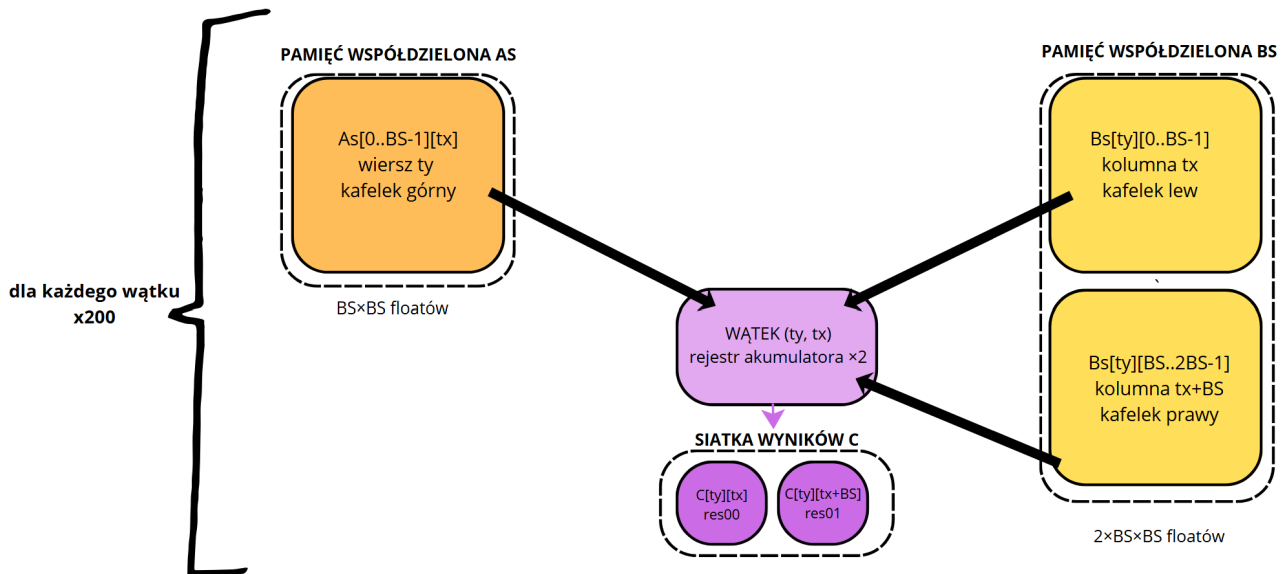
Wariant 2: Obliczanie 2 wyników przez wątek

Oznaczenie skrótowe: 2-Res

W tym podejściu pojedynczy wątek oblicza dwa sąsiednie wyniki wiersza macierzy docelowej ($C_{ty,tx}$ oraz $C_{ty,tx+BLOCK_SIZE}$). Wymaga to pobrania do pamięci współdzielonej jednego kafelka z macierzy A oraz dwóch kafelków z macierzy B. Dane przechowywane są wielokrotnie wykorzystywane w rejestrach wątku (`res1`, `res2`). Zgodnie z Listingiem 2, wątek wykonuje dodatkowy odczyt z macierzy B, co zwiększa intensywność obliczeniową przy tym samym koszcie ładowania danych z macierzy A.

```
1 // matrixMul.cu
2 As[ty][tx] = A[a + wA * ty + tx];
3 Bs[ty][tx] = B[b + wB * ty + tx];
4 Bs[ty][tx + BLOCK_SIZE] = B[b + wB * ty + tx + BLOCK_SIZE];
```

Listing 2: Fragment jądra obliczającego 2 wyniki na wątek.



Rysunek 2: Pobieranie danych do pamięci współdzielonej w wariantie 2-Res dla bloku 16×16 .

Wariant 3: Obliczanie 4 wyników przez wątek

Oznaczenie skrótowe: 4-Res

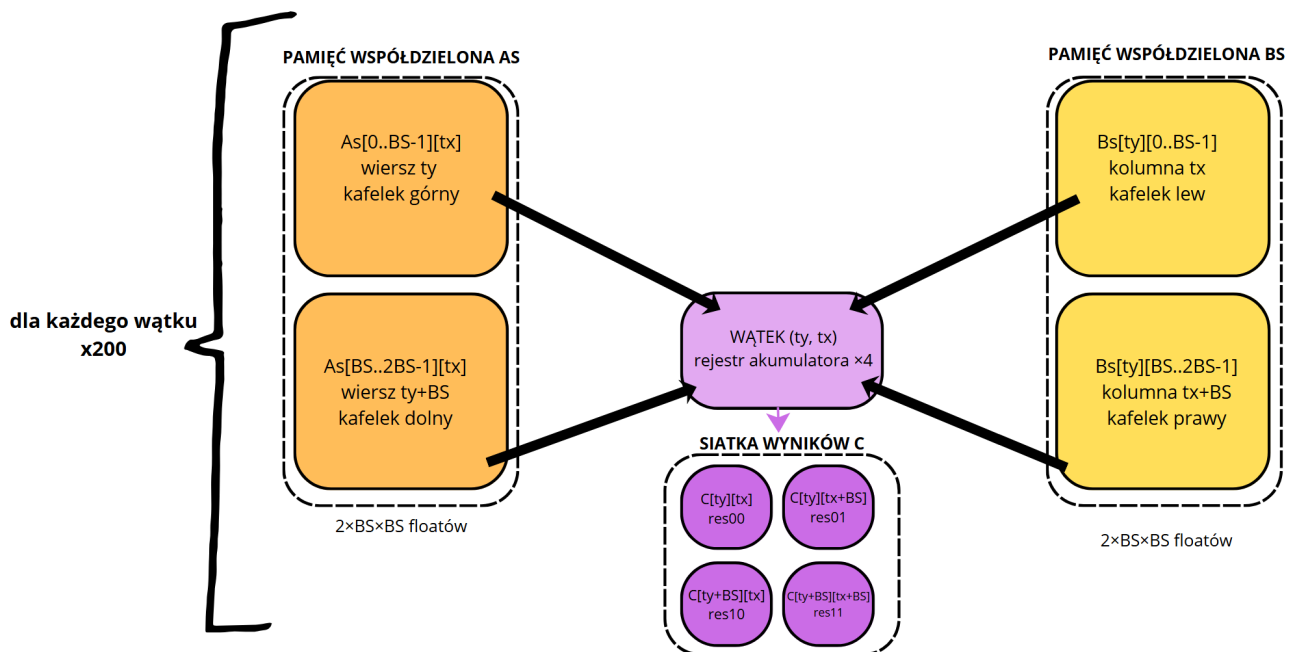
Wątek oblicza siatkę 2×2 wyników. W tym celu ładuje dwa kafelki z macierzy A oraz dwa z macierzy B. Skutkuje to maksymalnym ponownym użyciem danych w pamięci współdzielonej oraz koniecznością utrzymywania aż 4 wyników tymczasowych w rejestrach procesora. Wariant ten uruchomiono wyłącznie dla bloku 16×16 . Próba rozszerzenia tego wariantu do rozmiaru bloku 32×32 kończy się niepowodzeniem sprzętowym z powodu przekroczenia alokacji rejestrów, a nie limitu pamięci współdzielonej. Dla bloku 32×32 , wariant 4-Res wymagałby załadowania tablic $As[64][32]$ oraz $Bs[32][64]$, co zajmuje łącznie $(2048 + 2048) \times 4 \text{ B} = 16 \text{ KB}$ pamięci współdzielonej na blok. Mieści się to w limicie 48 KB. Jednakże wieloprocessor (SM) architektury Pascal posiada ścisły limit 65536 rejestrów 32-bitowych. Blok 32×32 to 1024 wątki. Aby zachować minimalną równoległość (np. 2 bloki na SM w celu ukrycia latencji szyny pamięci), na wątek może przypadać maksymalnie $65536/2048 = 32$ rejestry. Agresywne unrollowanie pętli w wariantcie 4-Res oraz konieczność jednoczesnego utrzymywania siatki rejestrów akumulatorów (`res00`, `res01`, `res10`, `res11`) wraz z indeksowaniem drastycznie przekracza ten limit, powodując błąd alokacji zasobów hardware'owych uniemożliwiający uruchomienie jądra. W kodzie widoczne jest pobieranie danych dla rozszerzonego obszaru roboczego wątku.

```

1 // matrixMul.cu
2 As[ty][tx] = A[a + wA * ty + tx];
3 As[ty + BLOCK_SIZE][tx] = A[a + wA * (ty + BLOCK_SIZE) + tx];
4 Bs[ty][tx] = B[b + wB * ty + tx];
5 Bs[ty][tx + BLOCK_SIZE] = B[b + wB * ty + tx + BLOCK_SIZE];

```

Listing 3: Fragment jądra obliczającego 4 wyniki na wątek.



Rysunek 3: Pobieranie danych do pamięci współdzielonej w wariantcie 4-Res dla bloku 16×16 .

Niejednorodne dane i test poprawności

Dla zapewnienia wiarygodności weryfikacji poprawności obliczeń zrezygnowano ze stałych wartości macierzy na rzecz danych niejednorodnych (zależnych od indeksów). Obliczenia GPU zweryfikowano względem wzorcowej funkcji wykonującej mnożenie na CPU. Ponieważ sprzętowe instrukcje GPU (FMA - Fused Multiply-Add) oraz inna kolejność sumowania powodują naturalny błąd precyzji zmiennoprzecinkowej dla dużych macierzy (3200×3200), test oparto na pomiarze błędu względnego ze zdefiniowaną tolerancją.

```

1 // Generacja niejednorodnych danych (wartosci pseudolosowe zalezne od indeksu)
2 data[i * w + j] = (i % 100) * 0.01f;
3
4 // Weryfikacja wyników przy pomocy błędu względnego
5 double cpu_ref = (double)h_C_ref[i];
6 double gpu_res = (double)h_C[i];
7 double rel_err = fabs(gpu_res - cpu_ref) / (fabs(cpu_ref) > 1.0 ? fabs(cpu_ref) : 1.0);
8
9 if (rel_err > 1e-2) {
10     correct = false; // Odrzucenie jeśli różnica względna przekracza 1%
11 }

```

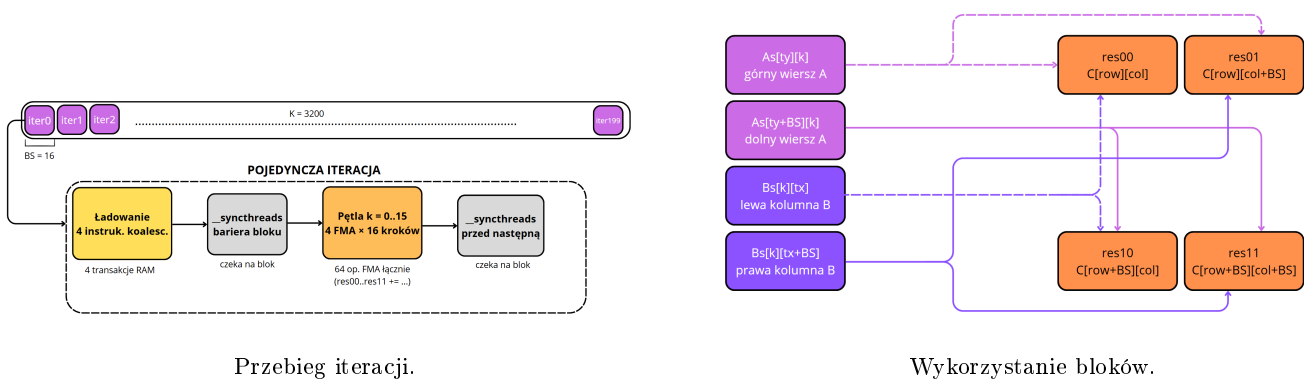
Listing 4: Logika testu poprawności z uwzględnieniem błędu względnego.

Wszystkie zaimplementowane warianty kerneli pomyślnie przeszły test walidacyjny, zgłaszając wynik PASS.

Obliczenia współczynnika CGMA (Teoria)

Współczynnik CGMA (Compute to Global Memory Access) określa stosunek liczby operacji arytmetycznych do liczby dostępow do pamięci globalnej. Na poziomie teoretycznym dla kafelkowego mnożenia macierzy:

- **Wariant 1-Res:** Każdy wątek ładuje 2 słowa (8 bajtów - jedno z A, jedno z B) z pamięci globalnej do współdzielonej, aby wykonać 1 operację FMA (2 operacje arytmetyczne: mnożenie i dodawanie) wewnątrz pętli k . Daje to teoretyczny stosunek 1 operacji na 1 załadowane słowo.
- **Wariant 2-Res:** Wątek pobiera do pamięci współdzielonej 3 słowa (1 słowo z macierzy A oraz 2 słowa z macierzy B), w oparciu o które wykonuje w pętli 2 operacje FMA (co odpowiada 4 operacjom arytmetycznym). Teoretyczny współczynnik CGMA wynosi w tym przypadku $\frac{4}{3} \approx 1.33$ operacji na jedno pobrane słowo z pamięci globalnej.
- **Wariant 4-Res:** Wątek ładuje 4 słowa (2 z A, 2 z B), ale wykonuje 4 operacje FMA (8 operacji arytmetycznych). Teoretyczny stosunek CGMA rośnie do 2 operacji na słowo, ponieważ dane raz załadowane do pamięci współdzielonej są wykorzystywane dwukrotnie częściej.



Rysunek 4: Przykład liczenia pojedynczej iteracji dla wariantu 4-Res.

3 Prezentacja wyników i omówienie przebiegu eksperymentu.

Obliczenia intensywności arytmetycznej z profilerem

Na podstawie danych zebranych przez profiler (`dram_read_bytes`, `dram_write_bytes`, `flop_count_sp`) wyliczono rzeczywistą intensywność arytmetyczną według wzoru:

$$\text{Intensywność} = \frac{\text{flop_count_sp}}{\text{dram_read_bytes} + \text{dram_write_bytes}} \left[\frac{\text{FLOP}}{\text{Byte}} \right]$$

Wariant	FLOPs	DRAM Read (B)	DRAM Write (B)	Intensywność
Oryginał (BS=16)	$6.55 \cdot 10^{10}$	$8.30 \cdot 10^9$	$4.16 \cdot 10^7$	7.85 FLOP/B
Oryginał (BS=32)	$6.55 \cdot 10^{10}$	$4.33 \cdot 10^9$	$4.11 \cdot 10^7$	15.00 FLOP/B
2-Res (BS=16)	$6.55 \cdot 10^{10}$	$8.27 \cdot 10^9$	$4.14 \cdot 10^7$	7.88 FLOP/B
2-Res (BS=32)	$6.55 \cdot 10^{10}$	$4.31 \cdot 10^9$	$4.14 \cdot 10^7$	15.05 FLOP/B
4-Res (BS=16)	$6.55 \cdot 10^{10}$	$4.21 \cdot 10^9$	$4.13 \cdot 10^7$	15.41 FLOP/B

Tablica 2: Intensywność arytmetyczna wyliczona na podstawie metryk profilera.

Jak wynika z powyższej tabeli, intensywność arytmetyczna rośnie wraz ze zwiększaniem pracy przypadającej na pojedynczy wątek. Wariant 4-Res (BS=16) osiąga intensywność porównywalną z wariantem oryginalnym dla bloku 32, co tłumaczy jego wysoką wydajność mimo mniejszego rozmiaru bloku.

Prędkość przetwarzania dla poszczególnych wariantów

Tabela 2 prezentuje zestawienie prędkości i czasu obliczeń w zależności od wariantu przydziału pracy oraz rozmiaru bloku wątków.

Wariant algorytmu	Blok 16×16	Czas [ms]	Blok 32×32	Czas [ms]
Oryginał (1-Res/wątek)	119.91 GFlop/s	546.54	165.95 GFlop/s	394.91
Zoptymalizowany (2-Res/wątek)	165.52 GFlop/s	395.94	188.08 GFlop/s	348.44
Maksymalny (4-Res/wątek)	211.22 GFlop/s	310.28	Brak (limit pam.)	-

Tablica 3: Porównanie prędkości obliczeń (GFlop/s) i czasu wykonania dla instancji 3200×3200 .

Mikro-uwagi do tabeli prędkości: Zauważalny jest trend drastycznego wzrostu przepustowości obliczeniowej w miarę zwiększania liczby wyników liczonych przez wątek. Obliczenia 4 wyników dla bloku 16×16 skutkują niemal dwukrotnym wzrostem wydajności względem oryginalnego algorytmu dla tego samego bloku.

Analiza metryk sprzętowych (nvprof)

Aby wyjaśnić przyczynę różnic w czasach wykonania poszczególnych wariantów, zebrano dane dotyczące ruchu w hierarchii pamięci karty graficznej oraz efektywności wykorzystania potoku wykonawczego.

Nazwa wariantu i blok	Occupancy	SM Efficiency	IPC	Global Load (Mld)	Shared Load (Mld)
Oryginał (BS=16)	99.9%	99.9%	1.05	2,048	1,536
Oryginał (BS=32)	99.8%	99.9%	1.53	1,024	1,536
2-Res (BS=16)	99.8%	99.9%	1.48	1,536	1,280
2-Res (BS=32)	99.8%	99.7%	1.69	0,768	1,280
4-Res (BS=16)	74.7%	99.8%	1.90	1,024	0,768

Tablica 4: Wyniki analizy przyczyn efektywności z wykorzystaniem profilera nvprof.

Mikro-uwagi do metryk sprzętowych:

- **Redukcja transakcji z pamięcią:** Zwiększenie krotności użycia raz pobranych danych drastycznie redukuje żądania dostępu do pamięci globalnej (*Global Load*). Algorytm wariantu 4-Res zredukował transakcje o 50% (z 2,04 mld do 1,02 mld) względem oryginału, co ma decydujący wpływ na szybkość pracy kernela.
- **Przechowywanie danych w rejestrach i redukcja ruchu pamięci:** Wprowadzenie 4 wyników na wątek przeniosło obciążenie z pamięci współdzielonej (*Shared Load Transactions* spadło o połowę: z 1,53 mld do 0,76 mld) bezpośrednio na ultraszybki plik rejestrów. Analiza teoretyczna przesyłu danych wskazuje, że dla instancji 3200×3200 przy kafelkowaniu bazowym wolumen żądań z pamięci globalnej powinien wynosić znacznie więcej, jednak wartość `gld_transactions` na poziomie 1,024 mld idealnie odwzorowuje redukcję transakcji szyny o 50% w stosunku do bazowego algorytmu BS=16 (2,048 mld). Różnice pomiędzy tą wartością a fizycznie

zmierzoną metryką `dram_read_bytes` w Tabeli 1 wynikają ze świetnego współczynnika trafień (*Hit Rate*) pamięci podręcznej L2, która przechwytuje powtarzające się odczyty pasm macierzy A i B, radykalnie zwiększając rzeczywistą intensywność arytmetyczną do poziomu **15.41 FLOP/B**.

- **Paradoks Zajętości (Occupancy vs. Performance):** Zwiększone zapotrzebowanie na rejestry w wariancie 4-Res (BS=16) wywołało spadek parametru *achieved_occupancy* do poziomu 74.7%. Multiprocessor Pascal SM pozwala na jednoczesne aktywowanie maksymalnie 2048 wątków (czyli 8 bloków po 256 wątków). Spadek occupancy do ok. 75% oznacza ograniczenie liczby aktywnych bloków z 8 do 6 na jeden SM, wymuszone wyczerpaniem puli rejestrów przez rozbudowany stan wewnętrzny jądra. Eksperyment dowodzi jednak, że wysoka zajętość (*occupancy*) nie jest warunkiem koniecznym do osiągnięcia maksymalnej wydajności. Dzięki drastycznemu podniesieniu poziomu niezależności instrukcji (ILP - *Instruction-Level Parallelism*) w pętli obliczeniowej, jądro 4-Res osiąga najwyższy wskaźnik IPC (1.90), skutecznie maskując latencję dostępu do pamięci pracą na rejestrach i osiągając rekordową wydajność **211.22 GFlop/s**.

4 Podsumowanie i wnioski.

Przeprowadzone badania optymalizacji kafelkowego algorytmu mnożenia macierzy w środowisku CUDA pozwoliły na szczegółową analizę kompromisów (ang. trade-offs) pomiędzy wielkością bloku wątków, intensywnością obliczeniową a fizycznymi ograniczeniami zasobów sprzętowych architektury Pascal (NVIDIA GeForce GTX 1080). Dobór rozmiaru instancji testowej (3200×3200) okazał się w pełni uzasadniony - zapewnił ciągle nasycenie multiprocessorów (SM) pracą, minimalizując błędy pomiarowe na styku wykonywania kernela. Wszystkie zaimplementowane warianty algorytmu pomyślnie przeszły testy walidacyjne z uwzględnieniem tolerancji błędu względnego dla obliczeń zmiennoprzecinkowych, co potwierdza ich poprawność. Głównym i najważniejszym wnioskiem płynącym z eksperymentu jest fakt, że maksymalizacja wskaźnika zajętości GPU (*achieved occupancy*) nie jest warunkiem koniecznym do osiągnięcia najwyższej wydajności obliczeniowej. Tezę tę doskonale ilustruje wariant 4-Res (BS=16). Pomimo spadku parametru occupancy do poziomu 74.7% (wymuszonego wysokim zużyciem rejestrów na wątek i ograniczeniem liczby aktywnych bloków na SM z 8 do 6), wariant ten osiągnął najwyższą przepustowość obliczeniową wynoszącą 211.22 GFlop/s oraz najkrótszy czas wykonania (310.28 ms). Kluczem do sukcesu okazało się drastyczne podniesienie równoległości na poziomie instrukcji (ILP) oraz przeniesienie operacji z pamięci współdzielonej bezpośrednio na ultraszybki plik rejestrów. Spowodowało to wzrost wskaźnika IPC do rekordowej wartości 1.90 oraz zredukowało liczbę transakcji Global Load aż o 50

Eksperyment ujawnił jednak sztywne **limity sprzętowe**, z którymi musi mierzyć się programista systemów równoległych:

- Zwiększanie intensywności arytmetycznej przynosi wymierne korzyści, dopóki zasoby sprzętowe nie zostaną wyczerpane.
- Próba uruchomienia wariantu 4-Res dla bloku 32×32 zakończyła się niepowodzeniem. Mimo że zapotrzebowanie na pamięć współdzieloną (16 KB) mieściło się w limicie 48 KB, agresywne unrollowanie pętli oraz rozbudowany stan wewnętrzny jądra doprowadziły do przekroczenia alokacji rejestrów (maksymalnie 32 rejestry na wątek przy chęci zachowania minimalnej równoległości) i błędów zasobów hardware'owych.